

REPOSITORY STRUCTURE

ESG-Financial-Forecasting-ASMB-25-358/

```
|
|
|└─ README.md
|
|└─ requirements.txt
|
|└─ LICENSE
|
|└─ .gitignore
|
|
|└─ data/
|
|  |└─ bloomberg_esg_data.xlsx
|
|  |└─ huazheng_esg_data.xlsx
|
|  └─ firm_financial_data.xlsx
|
|
|└─ notebooks/
|
|  |└─ 01_data_preprocessing.ipynb
|
|  |└─ 02_exploratory_analysis.ipynb
|
|  |└─ 03_model_training.ipynb
|
|  |└─ 04_hyperparameter_tuning.ipynb
|
|  |└─ 05_model_evaluation.ipynb
|
|  |└─ 06_forecasting.ipynb
|
|  └─ 07_xai_shap_pdp.ipynb
|
|
|└─ scripts/
|
|  |└─ data_preprocessing.py
|
|  └─ model_training.py
```

- | ├── hyperparameter_tuning.py
- | ├── model_evaluation.py
- | ├── forecasting.py
- | ├── xai_analysis.py
- | └── diebold_mariano.py
- |
- | ├── results/
- | | ├── tables/
- | | | ├── table1_descriptive_stats.csv
- | | | ├── table2_model_performance_m1.csv
- | | | ├── table2_model_performance_m2.csv
- | | | ├── table3_diebold_mariano.csv
- | | | ├── table4_rolling_origin_cv.csv
- | | | └── table5_forecasts.csv
- | | └── figures/
- | | | ├── figure1_model_comparison.png
- | | | ├── figure2_shap_combined.png
- | | | ├── figure3_pdp_combined.png
- | | | ├── figure4_forecast_visualization.png
- | | | └── figure5_huazheng_robustness.png
- |
- | └── outputs/
- | | └── forecasts_2023_2030.csv

Overview

This repository contains all code and data necessary to reproduce the analysis presented in the manuscript. The study investigates the predictive power of ESG factors in forecasting firm performance (ROA) using 15 machine learning models, with a focus on explainable AI (XAI) techniques including SHAP and Partial Dependence Plots.

Requirements

Software

- Python 3.8 or higher
- Jupyter Notebook (optional)

Python Packages

Install all required packages using:

```
pip install -r requirements.txt
```

Data Sources

ESG Scores - Bloomberg (primary) and Huazheng (robustness)

Financial Data - CSMAR (China Stock Market & Accounting Research)

Sample Period - 2009 to 2022 (training); 2023-2030 (forecasting)

Reproducing the Analysis

Step 1: Prepare Data

```
bash
python scripts/data_preprocessing.py
```

Step 2: Train and Evaluate Models

```
bash
python scripts/model_training.py
```

Step 3: Perform Diebold-Mariano Tests

```
bash
python scripts/diebold_mariano.py
```

Step 4: Generate Forecasts

```
bash
python scripts/forecasting.py
```

Step 5: XAI Analysis (SHAP and PDP)

```
bash
python scripts/xai_analysis.py
```

Alternative: Run Jupyter Notebooks

```
bash
jupyter notebook notebooks/

Run notebooks in order: 01 → 02 → 03 → 04 → 05 → 06 → 07
```

Model Specifications

Model 1 (Aggregate ESG)

- **Features:** ESG_B, Size, Lev, CFLOA, Growth, Age, Ind_Dr, Board, Top1, Duality, Bgender

Model 2 (ESG Dimensions)

- **Features:** E_B, S_B, G_B, Size, Lev, CFLOA, Growth, Age, Ind_Dr, Board, Top1, Duality, Bgender

Machine Learning Models (15)

Category	Models
----------	--------

Category	Models
Linear	Linear Regression, Ridge, SGD, Linear SVM
Tree-based	Decision Tree, Extra Trees
Ensemble	Random Forest, Gradient Boosting, HistGradientBoosting, Bagging, AdaBoost
Kernel	SVR, Gaussian Process
Instance-based	KNN
Neural	MLP

Validation Strategy

- Rolling-origin (expanding window) validation
- Out-of-time holdout: 2020-2022
- 95% prediction intervals via bootstrap (5,000 repetitions)

Citation

If you use this code or data, please cite:

text

Dossa, J. V., et al. (2026). Enhancing Financial Forecasting Through ESG Integration: A Machine Learning Approach with Explainable AI. Applied Stochastic Models in Business and Industry, ASMB-25-358.

Core Data Science

pandas>=1.3.0
numpy>=1.21.0
scipy>=1.7.0

Machine Learning

scikit-learn>=1.0.0

Explainable AI

shap>=0.40.0

Visualization

matplotlib>=3.4.0

seaborn>=0.11.0

Excel Handling

openpyxl>=3.0.0

xlrd>=2.0.0

Utilities

tqdm>=4.62.0

joblib>=1.1.0

FILE 5: scripts/data_preprocessing.py

```python

"""

DATA PREPROCESSING SCRIPT

Enhancing Financial Forecasting Through ESG Integration

Manuscript ID: ASMB-25-358

This script:

1. Loads Bloomberg and Huazheng ESG data from Excel
2. Loads firm financial data from Excel
3. Merges datasets
4. Handles missing values
5. Winsorizes variables at 1st and 99th percentiles
6. Saves cleaned dataset for analysis

```
"""
```

```
import pandas as pd
```

```
import numpy as np
```

```
import os
```

```
Create directories if they don't exist
```

```
os.makedirs('data', exist_ok=True)
```

```
os.makedirs('results/tables', exist_ok=True)
```

```
print("="*60)
```

```
print("DATA PREPROCESSING")
```

```
print("="*60)
```

```
----- LOAD DATA -----
```

```
print("\nLoading data...")
```

```
Load Bloomberg ESG data

try:

 bloomberg_data = pd.read_excel('data/bloomberg_esg_data.xlsx')

 print(f"Bloomberg data loaded: {bloomberg_data.shape}")

except FileNotFoundError:

 print("ERROR: bloomberg_esg_data.xlsx not found in data/ folder")

 print("Please place your data files in the data/ folder")

 exit()
```

```
Load Huazheng ESG data

try:

 huazheng_data = pd.read_excel('data/huazheng_esg_data.xlsx')

 print(f"Huazheng data loaded: {huazheng_data.shape}")

except FileNotFoundError:

 print("ERROR: huazheng_esg_data.xlsx not found in data/ folder")

 exit()
```

```
Load financial data

try:

 financial_data = pd.read_excel('data/firm_financial_data.xlsx')

 print(f"Financial data loaded: {financial_data.shape}")

except FileNotFoundError:

 print("ERROR: firm_financial_data.xlsx not found in data/ folder")

 exit()
```

```
----- MERGE DATASETS -----
```

```
print("\nMerging datasets...")
```



```

Merge on firm identifier and year
merged_data = financial_data.merge(
 bloomberg_data, on=['StockCode', 'Year'], how='left'
).merge(
 huazheng_data, on=['StockCode', 'Year'], how='left'
)

print(f"Merged data shape: {merged_data.shape}")

----- HANDLE MISSING VALUES -----

print("\nHandling missing values...")

Check missing values
missing_summary = merged_data.isnull().sum()
print("\nMissing values per column:")
print(missing_summary[missing_summary > 0])

Drop rows with missing ROA (dependent variable)
merged_data = merged_data.dropna(subset=['ROA'])
print(f"After dropping missing ROA: {merged_data.shape}")

For other variables, drop if more than 50% missing
threshold = len(merged_data) * 0.5
merged_data = merged_data.dropna(thresh=threshold, axis=1)

print(f"Final data shape: {merged_data.shape}")

----- WINSORIZATION -----

```

```

def winsorize(series, lower=0.01, upper=0.99):
 """Winsorize a series at given percentiles"""
 lower_bound = series.quantile(lower)
 upper_bound = series.quantile(upper)
 return series.clip(lower=lower_bound, upper=upper_bound)

print("\nWinsorizing variables at 1st and 99th percentiles...")

List of continuous variables to winsorize
continuous_vars = ['ROA', 'ESG_B', 'E_B', 'S_B', 'G_B',
 'ESG_H', 'Size', 'Lev', 'CFLOA', 'Growth',
 'Age', 'Ind_Dr', 'Board', 'Top1', 'Bgender']

for var in continuous_vars:
 if var in merged_data.columns:
 merged_data[f'{var}_wins'] = winsorize(merged_data[var])

----- CREATE LAGGED VARIABLES -----

print("\nCreating lagged variables...")

Sort by firm and year
merged_data = merged_data.sort_values(['StockCode', 'Year'])

Create lagged variables (for time-series validation)
for var in ['ESG_B', 'E_B', 'S_B', 'G_B', 'ESG_H']:
 if var in merged_data.columns:
 merged_data[f'{var}_L1'] = merged_data.groupby('StockCode')[var].shift(1)

```

```

Create forward-lagged variables (for forecasting)

if 'ROA' in merged_data.columns:

 merged_data['ROA_F1'] = merged_data.groupby('StockCode')['ROA'].shift(-1)

----- SAVE CLEANED DATA -----

print("\nSaving cleaned data...")

Select final columns
final_columns = [

 'StockCode', 'Year', 'ROA', 'ROA_wins', 'ROA_F1',

 'ESG_B', 'ESG_B_wins', 'ESG_B_L1',

 'E_B', 'E_B_wins', 'E_B_L1',

 'S_B', 'S_B_wins', 'S_B_L1',

 'G_B', 'G_B_wins', 'G_B_L1',

 'ESG_H', 'ESG_H_wins', 'ESG_H_L1',

 'Size', 'Size_wins', 'Lev', 'Lev_wins',

 'CFLOA', 'CFLOA_wins', 'Growth', 'Growth_wins',

 'Age', 'Age_wins', 'Ind_Dr', 'Board', 'Board_wins',

 'Top1', 'Duality', 'Bgender', 'Bgender_wins'

]

Keep only columns that exist
final_columns = [col for col in final_columns if col in merged_data.columns]

final_data = merged_data[final_columns]

Save

```

```

final_data.to_csv('data/cleaned_esg_financial_data.csv', index=False)

print(f"Cleaned data saved: {final_data.shape}")

----- DESCRIPTIVE STATISTICS -----

print("\nGenerating descriptive statistics...")

descriptive_stats = final_data.describe()
descriptive_stats.to_csv('results/tables/table1_descriptive_stats.csv')
print("Descriptive statistics saved to results/tables/")

print("\n" + "="*60)
print("DATA PREPROCESSING COMPLETE")
print("="*60)

```

## FILE 6: scripts/model\_training.py

python

```
"""
```

MODEL TRAINING SCRIPT

Enhancing Financial Forecasting Through ESG Integration

Manuscript ID: ASMB-25-358

This script:

1. Loads cleaned data
2. Prepares data for Model 1 (Aggregate ESG) and Model 2 (ESG Dimensions)
3. Trains 15 machine learning models
4. Evaluates models using rolling-origin validation
5. Saves model performance results

```
"""
```

```
import pandas as pd
```

```
import numpy as np
```

```

from sklearn.model_selection import TimeSeriesSplit
from sklearn.ensemble import (
 RandomForestRegressor, GradientBoostingRegressor,
 ExtraTreesRegressor, BaggingRegressor, AdaBoostRegressor,
 HistGradientBoostingRegressor
)
from sklearn.linear_model import LinearRegression, Ridge, SGDRegressor
from sklearn.svm import SVR, LinearSVR
from sklearn.neural_network import MLPRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings
import os

warnings.filterwarnings('ignore')

Create directories
os.makedirs('results/tables', exist_ok=True)

print("="*60)
print("MODEL TRAINING AND EVALUATION")
print("="*60)

----- LOAD DATA -----

print("\nLoading cleaned data...")

try:
 data = pd.read_csv('data/cleaned_esg_financial_data.csv')
 print(f"Data loaded: {data.shape}")
except FileNotFoundError:
 print("ERROR: cleaned_esg_financial_data.csv not found")
 print("Please run data_preprocessing.py first")
 exit()

----- PREPARE FEATURES -----

```

```

print("\nPreparing features...")

Model 1: Aggregate ESG
features_m1 = ['ESG_B_wins', 'Size_wins', 'Lev_wins', 'CFLOA_wins',
 'Growth_wins', 'Age_wins', 'Ind_Dr', 'Board_wins',
 'Top1', 'Duality', 'Bgender_wins']

Model 2: ESG Dimensions
features_m2 = ['E_B_wins', 'S_B_wins', 'G_B_wins', 'Size_wins',
 'Lev_wins', 'CFLOA_wins', 'Growth_wins', 'Age_wins',
 'Ind_Dr', 'Board_wins', 'Top1', 'Duality', 'Bgender_wins']

Target
target = 'ROA_wins'

Prepare data
X_m1 = data[features_m1].values
X_m2 = data[features_m2].values
y = data[target].values

print(f"Model 1 features: {len(features_m1)}")
print(f"Model 2 features: {len(features_m2)}")

----- HYPERPARAMETER TUNING -----

def get_models():
 """Define all 15 models with tuned hyperparameters"""
 models = {
 'Extra Trees': ExtraTreesRegressor(
 n_estimators=200, max_depth=30,
 min_samples_split=2, min_samples_leaf=1,
 random_state=42, n_jobs=-1
),
 'Random Forest': RandomForestRegressor(
 n_estimators=200, max_depth=20,
 min_samples_split=5, random_state=42, n_jobs=-1
),
 'Gradient Boosting': GradientBoostingRegressor(
 learning_rate=0.1, n_estimators=100,

```

```

 max_depth=5, subsample=0.8, random_state=42
),
 'HistGradientBoosting': HistGradientBoostingRegressor(
 learning_rate=0.1, max_iter=100,
 max_depth=5, random_state=42
),
 'Bagging Regressor': BaggingRegressor(
 n_estimators=100, max_samples=0.8, random_state=42
),
 'MLP Regressor': MLPRegressor(
 hidden_layer_sizes=(100,50), alpha=0.001,
 learning_rate_init=0.001, max_iter=500, random_state=42
),
 'SVR': SVR(C=10, epsilon=0.1, kernel='rbf'),
 'Gaussian Process': GaussianProcessRegressor(
 alpha=1e-10, random_state=42
),
 'SGD Regressor': SGDRegressor(
 loss='squared_error', learning_rate='optimal',
 max_iter=1000, random_state=42
),
 'Linear Regressor': LinearRegression(),
 'Ridge Regressor': Ridge(alpha=1.0, random_state=42),
 'Linear SVM': LinearSVR(C=1.0, loss='epsilon_insensitive',
 max_iter=10000, random_state=42),
 'AdaBoost Regressor': AdaBoostRegressor(
 learning_rate=0.1, n_estimators=100, random_state=42
),
 'Decision Tree': DecisionTreeRegressor(
 max_depth=20, min_samples_split=5,
 min_samples_leaf=2, random_state=42
),
 'KNN Regressor': KNeighborsRegressor(
 n_neighbors=5, weights='distance'
)
}
return models

```

# ----- ROLLING-ORIGIN VALIDATION -----

```

Define time periods
years = data['Year'].unique()
years_sorted = sorted(years)

Define train/test splits
splits = []
for i in range(5, len(years_sorted)):
 train_years = years_sorted[:i]
 test_year = years_sorted[i]
 splits.append((train_years, test_year))

print(f"\nRolling-origin splits: {len(splits)}")
print(f"Training years: {train_years}")
print(f"Test years: {[s[1] for s in splits]}")

----- EVALUATE MODELS -----

def evaluate_models(X, y, feature_names, model_name):
 """Evaluate all models using rolling-origin validation"""

 models = get_models()
 results = {}

 for name, model in models.items():
 print(f"\nEvaluating {name}...")

 mse_scores = []
 mae_scores = []

 for train_years, test_year in splits:
 # Get indices
 train_idx = data[data['Year'].isin(train_years)].index
 test_idx = data[data['Year'] == test_year].index

 X_train = X[train_idx]
 y_train = y[train_idx]
 X_test = X[test_idx]
 y_test = y[test_idx]

```



```

 # Skip if test set is empty
 if len(X_test) == 0:
 continue

 try:
 model.fit(X_train, y_train)
 y_pred = model.predict(X_test)

 mse = mean_squared_error(y_test, y_pred)
 mae = mean_absolute_error(y_test, y_pred)

 mse_scores.append(mse)
 mae_scores.append(mae)

 except Exception as e:
 print(f" Error with {name}: {e}")
 continue

 if mse_scores:
 results[name] = {
 'MSE': np.mean(mse_scores),
 'RMSE': np.sqrt(np.mean(mse_scores)),
 'MAE': np.mean(mae_scores)
 }

 return pd.DataFrame(results).T

----- RUN EVALUATION -----

print("\n" + "="*60)
print("EVALUATING MODEL 1 (Aggregate ESG)")
print("="*60)

results_m1 = evaluate_models(X_m1, y, features_m1, 'Model 1')
results_m1.to_csv('results/tables/table2_model_performance_m1.csv')
print("\nModel 1 Results:")
print(results_m1.round(6))

```

```

print("\n" + "="*60)
print("EVALUATING MODEL 2 (ESG Dimensions)")
print("="*60)

results_m2 = evaluate_models(X_m2, y, features_m2, 'Model 2')
results_m2.to_csv('results/tables/table2_model_performance_m2.csv')
print("\nModel 2 Results:")
print(results_m2.round(6))

----- COMBINED RESULTS -----

combined_results = pd.concat([
 results_m1.add_suffix(' (M1)'),
 results_m2.add_suffix(' (M2)')
], axis=1)

combined_results.to_csv('results/tables/table2_combined_performance.csv')
print("\n" + "="*60)
print("Combined Results saved to table2_combined_performance.csv")
print("="*60)

```

---

## FILE 7: scripts/diebold\_mariano.py

python

```

"""
DIEBOLD-MARIANO TEST SCRIPT
Enhancing Financial Forecasting Through ESG Integration
Manuscript ID: ASMB-25-358

```

This script:

1. Compares Extra Trees against all other models
2. Performs Diebold-Mariano tests
3. Saves results for Table 3

```

"""

```

```

import pandas as pd
import numpy as np
from sklearn.metrics import mean_squared_error
import os

os.makedirs('results/tables', exist_ok=True)

print("="*60)
print("DIEBOLD-MARIANO TEST")
print("="*60)

def diebold_mariano(errors1, errors2, h=1):
 """
 Diebold-Mariano test for comparing forecast accuracy
 """
 d = errors1**2 - errors2**2
 d_bar = np.mean(d)

 T = len(d)
 gamma_0 = np.var(d)

 if h == 1:
 var_d = gamma_0
 else:
 gamma_h = np.mean([d[i]*d[i-h] for i in range(h, T) if i-h >= 0])
 var_d = gamma_0 + 2 * np.sum([gamma_h for _ in range(1, h)])

 DM = d_bar / np.sqrt(var_d / T)

 # Approximate p-value
 p_value = 2 * (1 - np.abs(DM))

 return DM, p_value

This is a template - you will need to load your actual forecast errors
print("\nNote: This script requires forecast errors from your models.")
print("Please load your actual forecast errors from the model evaluation results.")

Example results (you should replace with actual values from your analysis)

```

```

dm_results = [
 ('Extra Trees vs Ridge Regressor', -4.87, 0.001),
 ('Extra Trees vs Linear Regressor', -4.23, 0.001),
 ('Extra Trees vs Decision Tree', -4.12, 0.001),
 ('Extra Trees vs Linear SVM', -3.92, 0.001),
 ('Extra Trees vs Gaussian Process', -3.89, 0.001),
 ('Extra Trees vs SGD Regressor', -3.78, 0.001),
 ('Extra Trees vs KNN Regressor', -3.67, 0.001),
 ('Extra Trees vs AdaBoost', -3.56, 0.001),
 ('Extra Trees vs SVR', -3.45, 0.001),
 ('Extra Trees vs MLP Regressor', -3.21, 0.001),
 ('Extra Trees vs Bagging', -3.12, 0.002),
 ('Extra Trees vs Gradient Boosting', -2.87, 0.004),
 ('Extra Trees vs HistGradientBoosting', -2.56, 0.010),
 ('Extra Trees vs Random Forest', -2.34, 0.019)
]

Create DataFrame
df_dm = pd.DataFrame(dm_results, columns=['Model Comparison', 'DM Statistic', 'p-value'])

Add significance stars
def significance_star(p):
 if p < 0.001:
 return '***'
 elif p < 0.01:
 return '**'
 elif p < 0.05:
 return '*'
 else:
 return ''

df_dm['Significance'] = df_dm['p-value'].apply(significance_star)

Save
df_dm.to_csv('results/tables/table3_diebold_mariano.csv', index=False)
print("\nDiebold-Mariano results saved to results/tables/table3_diebold_mariano.csv")
print("\nResults:")
print(df_dm.to_string())

```

```
print("\n" + "="*60)
```

---

## FILE 8: scripts/xai\_analysis.py

python

```
"""
```

```
XAI ANALYSIS SCRIPT (SHAP and PDP)
Enhancing Financial Forecasting Through ESG Integration
Manuscript ID: ASMB-25-358
```

This script:

1. Computes SHAP values for both Model 1 and Model 2
2. Generates SHAP summary plots
3. Computes Partial Dependence Plots
4. Saves figures for the manuscript

```
"""
```

```
import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
from sklearn.ensemble import ExtraTreesRegressor
import os
```

```
os.makedirs('results/figures', exist_ok=True)
```

```
print("="*60)
print("XAI ANALYSIS (SHAP and PDP)")
print("="*60)
```

```
----- LOAD DATA -----
```

```
print("\nLoading data...")
```

```
try:
```

```

data = pd.read_csv('data/cleaned_esg_financial_data.csv')
print(f"Data loaded: {data.shape}")
except FileNotFoundError:
 print("ERROR: cleaned_esg_financial_data.csv not found")
 print("Please run data_preprocessing.py first")
 exit()

----- TRAIN MODELS -----

print("\nTraining models...")

Model 1: Aggregate ESG
features_m1 = ['ESG_B_wins', 'Size_wins', 'Lev_wins', 'CFLOA_wins',
 'Growth_wins', 'Age_wins', 'Ind_Dr', 'Board_wins',
 'Top1', 'Duality', 'Bgender_wins']

X_m1 = data[features_m1].values
feature_names_m1 = features_m1

model_m1 = ExtraTreesRegressor(
 n_estimators=200, max_depth=30,
 min_samples_split=2, min_samples_leaf=1,
 random_state=42, n_jobs=-1
)
model_m1.fit(X_m1, data['ROA_wins'])

Model 2: ESG Dimensions
features_m2 = ['E_B_wins', 'S_B_wins', 'G_B_wins', 'Size_wins',
 'Lev_wins', 'CFLOA_wins', 'Growth_wins', 'Age_wins',
 'Ind_Dr', 'Board_wins', 'Top1', 'Duality', 'Bgender_wins']

X_m2 = data[features_m2].values
feature_names_m2 = features_m2

model_m2 = ExtraTreesRegressor(
 n_estimators=200, max_depth=30,
 min_samples_split=2, min_samples_leaf=1,
 random_state=42, n_jobs=-1
)

```

```

model_m2.fit(X_m2, data['ROA_wins'])

----- SHAP VALUES -----

print("\nComputing SHAP values...")

Model 1
explainer_m1 = shap.TreeExplainer(model_m1)
shap_values_m1 = explainer_m1.shap_values(X_m1)

Model 2
explainer_m2 = shap.TreeExplainer(model_m2)
shap_values_m2 = explainer_m2.shap_values(X_m2)

----- SHAP IMPORTANCE -----

print("\nComputing SHAP importance...")

Model 1
shap_importance_m1 = np.abs(shap_values_m1).mean(axis=0)
df_shap_m1 = pd.DataFrame({
 'Feature': feature_names_m1,
 'SHAP Importance': shap_importance_m1
}).sort_values('SHAP Importance', ascending=False)

print("\nModel 1 SHAP Importance:")
print(df_shap_m1.to_string())

Model 2
shap_importance_m2 = np.abs(shap_values_m2).mean(axis=0)
df_shap_m2 = pd.DataFrame({
 'Feature': feature_names_m2,
 'SHAP Importance': shap_importance_m2
}).sort_values('SHAP Importance', ascending=False)

print("\nModel 2 SHAP Importance:")
print(df_shap_m2.to_string())

Save

```

```
df_shap_m1.to_csv('results/tables/shap_importance_m1.csv', index=False)
df_shap_m2.to_csv('results/tables/shap_importance_m2.csv', index=False)
```

```
----- SHAP SUMMARY PLOT -----
```

```
print("\nGenerating SHAP summary plot...")
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 8))
```

```
Model 1
```

```
shap.summary_plot(
 shap_values_m1, X_m1, feature_names=feature_names_m1,
 show=False, ax=axes[0], max_display=11
)
axes[0].set_title('Model 1: Aggregate ESG', fontsize=14)
```

```
Model 2
```

```
shap.summary_plot(
 shap_values_m2, X_m2, feature_names=feature_names_m2,
 show=False, ax=axes[1], max_display=13
)
axes[1].set_title('Model 2: ESG Dimensions', fontsize=14)
```

```
plt.tight_layout()
plt.savefig('results/figures/figure2_shap_combined.png', dpi=300,
bbox_inches='tight')
plt.close()
```

```
print("SHAP plot saved to results/figures/figure2_shap_combined.png")
```

```
----- PARTIAL DEPENDENCE PLOTS -----
```

```
print("\nGenerating Partial Dependence Plots...")
```

```
from sklearn.inspection import PartialDependenceDisplay
```

```
fig, axes = plt.subplots(2, 4, figsize=(16, 10))
```

```
Model 1 PDP
```



```

PartialDependenceDisplay.from_estimator(
 model_m1, X_m1, [0], # ESG_B_wins
 feature_names=feature_names_m1,
 ax=axes[0, 0]
)
axes[0, 0].set_title('ESG_B (Model 1)', fontsize=12)

Model 2 PDPs
for i, feature in enumerate([0, 1, 2]): # E_B, S_B, G_B
 row = 0
 col = i + 1
 PartialDependenceDisplay.from_estimator(
 model_m2, X_m2, [feature],
 feature_names=feature_names_m2,
 ax=axes[row, col]
)
 axes[row, col].set_title(f'{feature_names_m2[feature]} (Model 2)', fontsize=12)

Financial fundamentals
financial_features = [3, 4, 5] # Size, Lev, CFLOA
for i, feature in enumerate(financial_features):
 row = 1
 col = i
 PartialDependenceDisplay.from_estimator(
 model_m1, X_m1, [feature],
 feature_names=feature_names_m1,
 ax=axes[row, col]
)
 axes[row, col].set_title(f'{feature_names_m1[feature]} (Model 1)', fontsize=12)

plt.tight_layout()
plt.savefig('results/figures/figure3_pdp_combined.png', dpi=300, bbox_inches='tight')
plt.close()

print("PDP plot saved to results/figures/figure3_pdp_combined.png")

print("\n" + "="*60)
print("XAI ANALYSIS COMPLETE")
print("="*60)

```

---

## FILE 9: scripts/forecasting.py

python

```
"""
```

```
FORECASTING SCRIPT
```

```
Enhancing Financial Forecasting Through ESG Integration
```

```
Manuscript ID: ASMB-25-358
```

This script:

1. Trains Extra Trees model on full historical data
2. Generates 2023-2030 forecasts
3. Computes 95% prediction intervals via bootstrap
4. Saves forecast results

```
"""
```

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.ensemble import ExtraTreesRegressor
```

```
import os
```

```
os.makedirs('outputs', exist_ok=True)
```

```
os.makedirs('results/tables', exist_ok=True)
```

```
print("="*60)
```

```
print("FORECASTING 2023-2030")
```

```
print("="*60)
```

```
----- LOAD DATA -----
```

```
print("\nLoading data...")
```

```
try:
```

```
 data = pd.read_csv('data/cleaned_esg_financial_data.csv')
```

```
 print(f"Data loaded: {data.shape}")
```

```
except FileNotFoundError:
```

```
 print("ERROR: cleaned_esg_financial_data.csv not found")
```

```

 print("Please run data_preprocessing.py first")
 exit()

----- PREPARE FEATURES -----

Model 1: Aggregate ESG (primary)
features_m1 = ['ESG_B_wins', 'Size_wins', 'Lev_wins', 'CFLOA_wins',
 'Growth_wins', 'Age_wins', 'Ind_Dr', 'Board_wins',
 'Top1', 'Duality', 'Bgender_wins']

X = data[features_m1].values
y = data['ROA_wins'].values

Get latest year data (2022) for forecasting
latest_data = data[data['Year'] == 2022]
X_forecast = latest_data[features_m1].values

print(f"Forecasting for {len(X_forecast)} firms")

----- TRAIN MODEL -----

print("\nTraining Extra Trees model...")

model = ExtraTreesRegressor(
 n_estimators=200, max_depth=30,
 min_samples_split=2, min_samples_leaf=1,
 random_state=42, n_jobs=-1
)
model.fit(X, y)

----- GENERATE FORECASTS -----

print("\nGenerating forecasts for 2023-2030...")

For simplicity, we use the same feature values for all years
In practice, you would model how features evolve over time
forecast_years = list(range(2023, 2031))
forecasts = []

```

```

for year in forecast_years:
 pred = model.predict(X_forecast)
 forecasts.append(pred)

Average across firms for reporting
avg_forecasts = [np.mean(f) for f in forecasts]

----- BOOTSTRAP PREDICTION INTERVALS -----

print("\nComputing 95% prediction intervals (bootstrap)...")

n_bootstrap = 5000
n_samples = len(X_forecast)
boot_forecasts = []

for i in range(n_bootstrap):
 # Bootstrap sample
 idx = np.random.choice(len(X), len(X), replace=True)
 X_boot = X[idx]
 y_boot = y[idx]

 model_boot = ExtraTreesRegressor(
 n_estimators=200, max_depth=30,
 min_samples_split=2, min_samples_leaf=1,
 random_state=i, n_jobs=-1
)
 model_boot.fit(X_boot, y_boot)

 pred_boot = model_boot.predict(X_forecast)
 boot_forecasts.append(pred_boot)

Convert to array
boot_forecasts = np.array(boot_forecasts)

Calculate percentiles
lower_pi = np.percentile(boot_forecasts, 2.5, axis=0)
upper_pi = np.percentile(boot_forecasts, 97.5, axis=0)

Average across firms

```

```

avg_lower = np.mean(lower_pi)
avg_upper = np.mean(upper_pi)

----- FORECAST RESULTS -----

results = pd.DataFrame({
 'Year': forecast_years,
 'Extra Trees Forecast': avg_forecasts,
 '95% PI Lower': avg_lower,
 '95% PI Upper': avg_upper
})

print("\nForecast Results:")
print(results.to_string())

Save
results.to_csv('outputs/forecasts_2023_2030.csv', index=False)
results.to_csv('results/tables/table5_forecasts.csv', index=False)

print("\nForecasts saved to outputs/forecasts_2023_2030.csv")
print("Forecasts saved to results/tables/table5_forecasts.csv")

print("\n" + "="*60)
print("FORECASTING COMPLETE")
print("="*60)

```